

Big Data Platforms

DATA140031 (MOOC, 3 ECTS) + DATA140032 (MOOC EXAM, 2 ECTS)

Lecture 1

Keijo Heljanko

Department of Computer Science
University of Helsinki
keijo.heljanko@helsinki.fi

1.9-2026



Big Data Platforms

DATA140031 (MOOC, 3 ECTS) +

DATA140032 (MOOC EXAM, 2 ECTS)

- ▶ Course homepage:
<https://big-data-platforms-26.mooc.fi/>
- ▶ This course focuses on advanced scalable cloud computing technologies and on key algorithmic ideas and methods used to implement them.
- ▶ Video Lectures by Zoom on Tuesdays 10:15-11:45 and on Thursdays 12:15-13:45 Finnish time.
- ▶ Lecture slides will be available, as well as a video recording of the lecture typically within 48 hours after the end of each lecture.



Course Requirements (MOOC, 3ECTS)

To pass the course DATA140031 (MOOC, 3 ECTS) you have to:

- ▶ Pass/Fail: Get enough points ($\geq 60\%$) from automatically graded Apache PySpark programming home exercises by the stated exercise deadlines.
- ▶ This part of the course is free of charge
- ▶ You will need basic programming skills in Python as well as basic database query language SQL skills in the home exercises. Be prepared to learn such skills if you don't already have them.
- ▶ The use of large language model based tools (e.g., ChatGPT) is also prohibited in this course.



Course Requirements (MOOC EXAM, 2ECTS)

To pass the course DATA140032 (MOOC EXAM, 2 ECTS) you have to:

- ▶ Pass the Exam, course graded 0-5
- ▶ This part is subject to Open University fees non non-UH students
- ▶ The Exam is planned to be in Examinarium (onsite in Helsinki)
- ▶ Exam will cover both the home assignments and the lectures
- ▶ UH students: You need to have both DATA140031 and DATA140032 passed to include the course in your CS or Data Science major studies



Course Registration for Both Courses

- ▶ UH students: Register in Sisu
- ▶ Non-UH students: Register through open University
- ▶ More details of registration to follow in upcoming weeks



Lecture Schedule

Lectures will be held on Zoom, see link on the course homepage

- ▶ Lecture 1: Tue 1.9.2026 at 10:15-11:45
- ▶ Lecture 2: Thu 3.9.2026 at 12:15-13:45
- ▶ Lecture 3: Tue 8.9.2026 at 10:15-11:45
- ▶ Lecture 4: Thu 10.9.2026 at 12:15-13:45
- ▶ Lecture 5: Tue 15.9.2026 at 10:15-11:45
- ▶ Lecture 6: Thu 17.9.2026 at 12:15-13:45
- ▶ Lecture 7: Tue 22.9.2026 at 10:15-11:45
- ▶ Lecture 8: Thu 24.9.2026 at 12:15-13:45
- ▶ Lecture 9: Thu 1.10.2026 at 12:15-13:45
- ▶ Lecture 10: Tue 6.10.2026 at 10:15-11:45
- ▶ Lecture 11: Tue 13.10.2026 at 10:15-11:45



Backup Lecture slots

There is a backup slots for lectures if we need to reschedule any of the lectures:

- ▶ Backup Lecture slot: Thu 15.10.2026 at 12:15-13:45



Home Exercise Schedule

The home exercises are done with a Docker based PySpark JupyterHub environment provided by the course. The assignments are automatically graded periodically, so expect some delays in getting your scores registered.

Schedule, exact deadline times always 16:00 Finnish time:

- ▶ 8.9 - Exercise 1 distributed, due date 22.9 at 16:00
- ▶ 15.9 - Exercise 2 distributed, due date 29.9 at 16:00
- ▶ 22.9 - Exercise 3 distributed, due date 6.10 at 16:00
- ▶ 29.9 - Exercise 4 distributed, due date 13.10 at 16:00
- ▶ 6.10 - Exercise 5 distributed, due date 20.10 at 16:00
- ▶ 13.10 - Extra Exercise 6 distributed, due date 31.10 at 16:00



Homework Exercise Points

We will use the following grading scale:

- ▶ The grading scale will be as follows. Minimum 60% from home exercise totals needed to pass the MOOC (3 ECTS) part of the course.
- ▶ The Extra round points from home exercises will count towards your home exercise points but will not allow home exercises contribution to go over 100%.



Course Personnel

- ▶ Lectures: Prof. Keijo Heljanko
 - ▶ Email: keijo.heljanko@helsinki.fi
- ▶ Home exercises: Haining Tong, Abdur Rahman, Ying Song, Andreas Niskanen, and Oskar Lappi
 - ▶ Discord channel for course:
<https://study.cs.helsinki.fi/discord/join/bdp>
 - ▶ Discord channel is the main announcement channel for the course.
 - ▶ Please use the Discord channel for questions about the home exercises.
 - ▶ Email of the course:
big-data-platforms-f26@helsinki.fi



Course Material

- ▶ Home exercises are to be done using an automated home exercise grading system. We are developing the system further every year, consider yourselves as beta testers of the system, so please be patient and expect some minor glitches.
- ▶ All course material will be distributed through the mooc.fi portal.
- ▶ Material will mostly consist of the lecture slides.
- ▶ There is no single book the course will be based on but we can recommend some additional reading on the topic.



Cloud Computing

- ▶ The NIST Definition of Cloud Computing

Authors: Peter Mell and Tim Grance

SP 800-145

<https://csrc.nist.gov/publications/detail/sp/800-145/final>

- ▶ Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. This cloud model promotes availability and is composed of five essential **characteristics**, three **service models**, and four **deployment models**.



Cloud Computing (cnt.)

- ▶ **Essential characteristics:** On-demand self-service, Broad network access, Resource pooling, Rapid elasticity, and Measured Service.
- ▶ **Service Models:** Cloud Software as a Service (SaaS), Cloud Platform as a Service (PaaS), and Cloud Infrastructure as a Service (IaaS).
- ▶ **Deployment Models:** Private cloud, Community cloud, Public cloud, and Hybrid cloud.



Business Drivers of Clouds

- ▶ Large data centers allow for economics of scale
 - ▶ Cheaper hardware purchases
 - ▶ Cheaper cooling of hardware
 - ▶ Example: Google paid 40 MEur for a Summa paper mill site in Hamina, Finland: Data center cooled with sea water from the Baltic Sea
 - ▶ Cheaper electricity
 - ▶ Cheaper network capacity
 - ▶ Smaller number of administrators / computer
- ▶ Unreliable commodity hardware is used
- ▶ Reliability obtained by replication of hardware components and a combined with a **fault tolerant software stack**



Cloud Computing Technologies

A collection of technologies aimed to provide elastic “pay as you go” computing

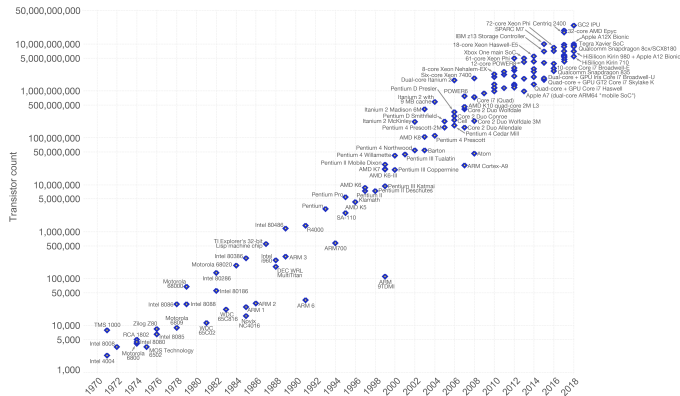
- ▶ **Virtualization of computing resources:** Amazon EC2&EKS, Microsoft Azure, Google Cloud Platform, Open Stack Compute, Kubernetes, . . .
- ▶ **Scalable file storage:** Amazon S3, GFS, HDFS, . . .
- ▶ **Scalable batch processing:** Google MapReduce / Apache Hadoop, Apache Flink, Apache Spark, . . .
- ▶ **Scalable datastore:** Amazon Dynamo, Apache Cassandra, Riak, Google Bigtable, HBase, Chubby, Apache Zookeeper. . .
- ▶ **Scalable Web applications hosting:** Google App Engine, Microsoft Azure, Heroku, . . .



Moore's Law

Moore's Law – The number of transistors on integrated circuit chips (1971-2018)

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are linked to Moore's law.



Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)

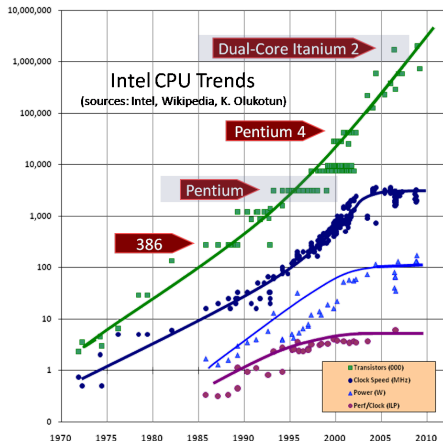
The data visualization is available at [OurWorldinData.org](https://www.ourworldindata.org). There you find more visualizations and research on this topic.

Licensed under CC-BY-SA by the author Max Roser.

- Gordon Moore of Intel 1965: the number of transistors in integrated circuits doubles roughly every 18-24 months



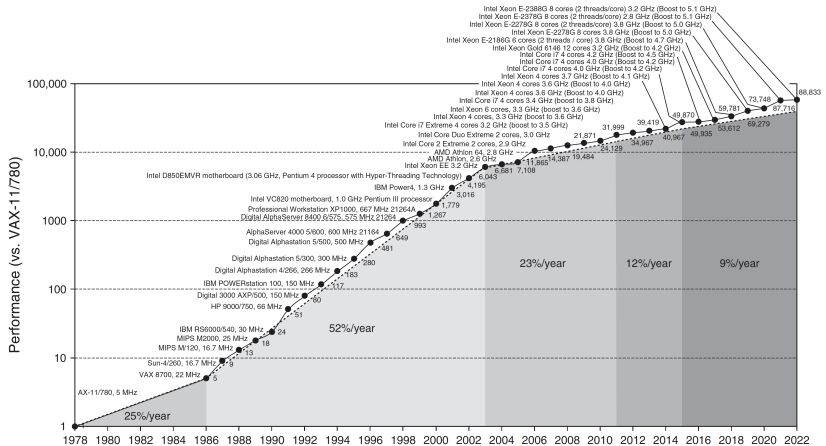
Clock Speed of Processors



- Herb Sutter: The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software. Dr. Dobb's Journal, 30(3), March 2005 (updated graph in August 2009).



Single Threaded Performance Improvements



- John Hennessy David Patterson: Computer Architecture: A Quantitative Approach, 7th ed., 2026



Implications of the End of Free Lunch

- ▶ The clock speeds of microprocessors are not going to improve much in the foreseeable future
 - ▶ The efficiency gains in single threaded performance are going to be only moderate
- ▶ The number of transistors in a microprocessor is still growing at a high rate
 - ▶ One of the main uses of transistors has been to increase the number of computing cores the processor has
 - ▶ The number of cores in a low end workstation (as those employed in large scale datacenters) is going to keep on steadily growing
- ▶ Programming models need to change to efficiently exploit all the available concurrency - scalability to high number of cores/processors will need to be a major focus



Dark Silicon - End of Moore's law in Sight?!

- ▶ Even worse news ahead, computing will be hitting a power wall: Silicon is becoming energy constrained
- ▶ We will have much more transistors than what we can switch on and off at each clock cycle: **Dark Silicon**
- ▶ For implications of dark silicon, see:
 - ▶ Hadi Esmaeilzadeh, Emily R. Blem, Renée St. Amant, Karthikeyan Sankaralingam, Doug Burger: Dark Silicon and the End of Multicore Scaling. IEEE Micro 32(3): 122-134 (2012)
 - ▶ Michael B. Taylor: A Landscape of the New Dark Silicon Design Regime. IEEE Micro 33(5): 8-19 (2013)



Dark Silicon - Improving Energy Efficiency

- ▶ To combat the power wall more energy efficient computing is needed
- ▶ GPGPU computing is improving the energy efficiency of training deep neural networks
- ▶ Special purpose hardware (ASICs) for inference of deep neural networks: **Google Tensor Processing Units**
- ▶ Similar special purpose accelerators will become a necessity also to other fields than machine learning



Amdahl's Law

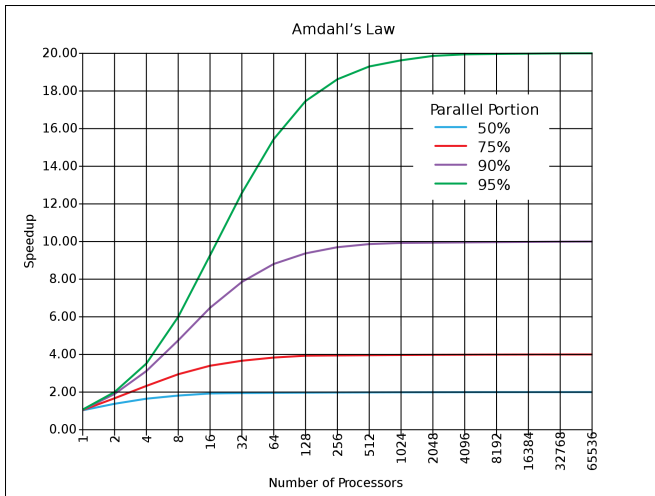
- ▶ In order to keep the ever increasing number of cores busy, the applications must be extremely parallel to enjoy the benefits of increasing transistor counts
- ▶ Unfortunately, the Amdahl's Law still applies:
 - ▶ Amdahl's law states that if P is the proportion of a program that can be made parallel (i.e. benefit from parallelization), and $(1 - P)$ is the proportion that cannot be parallelized (remains serial), then the maximum speedup that can be achieved by using N processors is:

$$\frac{1}{(1 - P) + \frac{P}{N}}$$

http://en.wikipedia.org/wiki/Amdahl%27s_law



Amdahl's Law (Example from Wikipedia)



Scaling Up vs Scaling Out

- ▶ **Scaling up:** When the need for parallelism arises, a single powerful computer is added with more CPU cores, more memory, and more hard disks
- ▶ **Scaling out:** When the need for parallelism arises, the task is divided between a large number of less powerful machines with (relatively) slow CPUs, moderate memory amounts, moderate hard disk counts
- ▶ **Big Data Platforms are trying to exploiting scaling out instead of scaling up**



Pros and Cons of Scaling Up vs Scaling Out

- ▶ **Scaling up is more expensive than scaling out.** Big high-end systems have much higher pricing for a given: CPU power, memory, and hard disk space
- ▶ **Scaling out is more challenging for fault tolerance.** A large number of loosely coupled systems means more components and thus more failures in hardware and in networking. **Solution: Software fault tolerance**
- ▶ **Scaling out is more challenging for software development** due to larger number of components, larger number of failures both in nodes and networking connecting them, and increased latencies. **Solution: Scalable Big Data Platforms**



Warehouse-scale Computing (WSC)

- ▶ The smallest unit of computation in Google scale is:
Warehouse full of computers
- ▶ [WSC]: Luiz André Barroso, Urs Hölzle, and Parthasarathy Ranganathan: *The Datacenter as a Computer: Designing Warehouse-Scale Machines, 4th edition*
<https://dx.doi.org/10.1007/978-3-031-99489-0>
- ▶ The WSC book motto:
“... we must treat the datacenter itself as one massive warehouse-scale computer (WSC).”



Google Summa Warehouse Scale Computer



Google Summa from the Roof Level



Google Summa Sea Water Cooling



Google Summa Computer Hall



Elements of WSC

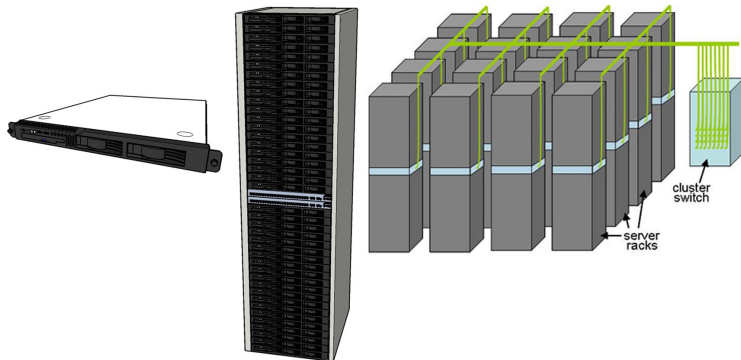


Figure: Typical elements in WSC, Figure 1.1 of [WSC, 2nd Ed.]



Google WSC from 2012

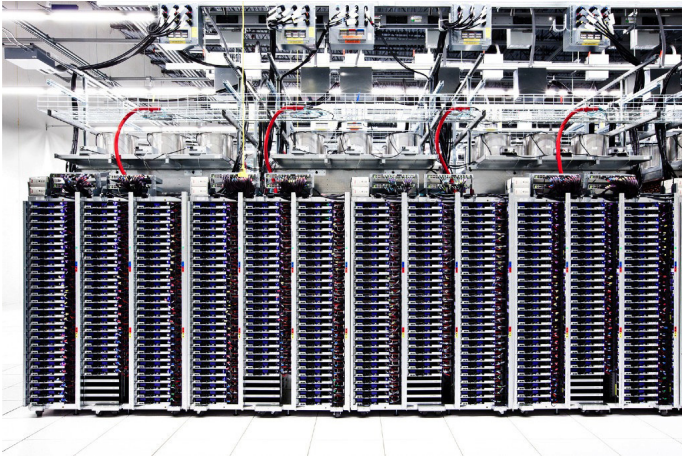


Figure: Picture of a row of servers from Google, Figure 1.2 of [WSC,2nd Ed.]



A Classical Datacenter

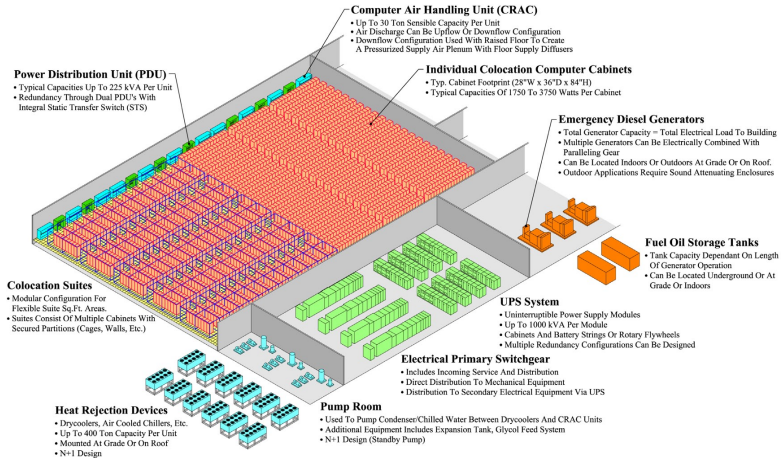


Figure: A Classical Datacenter, Figure 4.1 of [WSC, 2nd Ed.]



A Classical Cooling System

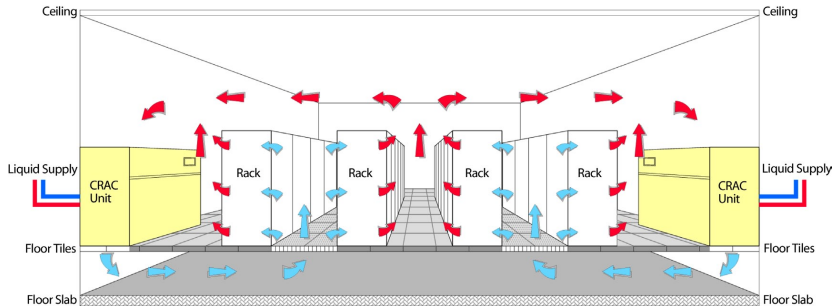


Figure: A Classical Cooling System, Figure 4.6 of [WSC, 2nd Ed.]



Power Usage Breakdown of Servers

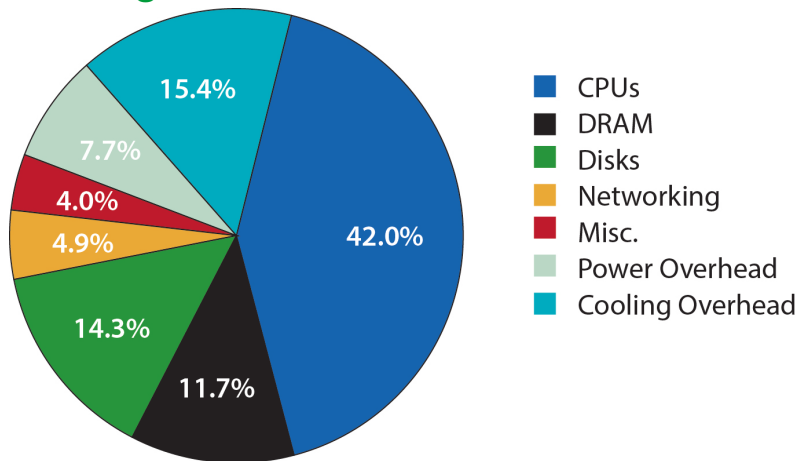


Figure: Power Usage Breakdown for Two Socket Servers (from 2012), Figure 1.6 of [WSC, 2nd Ed.]



Storage Hierarchy of WSC

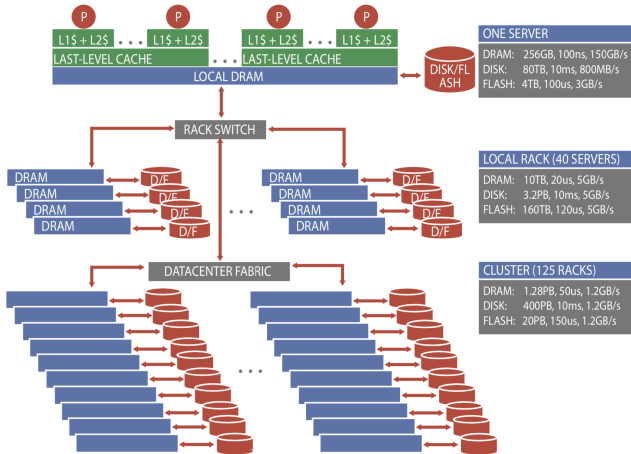


Figure: Storage hierarchy of WSC, Figure 3.15 of [WSC]



WSC latency, bandwidth, and capacity

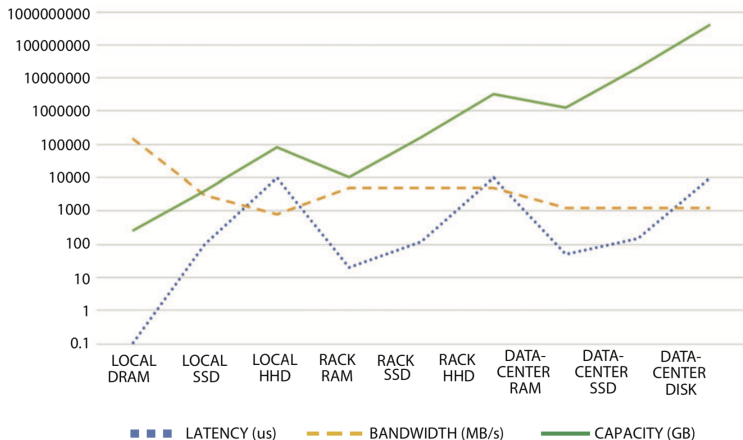


Figure: WSC latency, bandwidth, and capacity, Figure 3.16 of [WSC]



Jeffrey Dean (Google): Rule of Thumb Numbers

- LADIS 2009 keynote: “Designs, Lessons and Advice from Building Large Distributed Systems”

<http://www.cs.cornell.edu/projects/ladis2009/talks/dean-keynote-ladis2009.pdf>

L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	3,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA $\Rightarrow$ Netherlands $\Rightarrow$ CA	150,000,000 ns



Jeffrey Dean (Google): LADIS 2009 keynote failure numbers

- ▶ At warehouse-scale, failures will be the norm and thus **fault tolerant software will be inevitable**
- ▶ Hypothetically assume that server mean time between failures (MTBF) would be 30 years (highly optimistic and not realistic number!). In a WSC with 10000 nodes roughly one node will fail per day.
- ▶ **Typical yearly flakiness metrics from J. Dean (Google, 2009):**
 - ▶ 1-5% of your disk drives will die
 - ▶ Servers will crash at least twice (2-4% failure rate)



Jeffrey Dean (Google): Joys of Real Hardware (in verbatim)

Typical first year for a new cluster:

- ▶ ≈ 0.5 overheating (power down most machines in < 5 mins ≈ 1 -2 days to recover)
- ▶ ≈ 1 PDU failure (≈ 500 -1000 machines suddenly disappear, ≈ 6 hours to come back)
- ▶ ≈ 1 rack-move (plenty of warning, ≈ 500 -1000 machines powered down, ≈ 6 hours)
- ▶ ≈ 1 network rewiring (rolling $\approx 5\%$ of machines down over 2-day span)
- ▶ ≈ 20 rack failures (40-80 machines instantly disappear, 1-6 hours to get back)
- ▶ ≈ 5 racks go wonky (40-80 machines see 50% packetloss)
- ▶ ≈ 8 network maintenances (4 might cause ≈ 30 -minute random connectivity losses)



Jeffrey Dean (Google): Joys of Real Hardware (cnt.)

- ▶ $\approx$ 12 router reloads (takes out DNS and external vips for a couple minutes)
- ▶ $\approx$ 3 router failures (have to immediately pull traffic for an hour)
- ▶ $\approx$ dozens of minor 30-second blips for dns
- ▶ $\approx$ 1000 individual machine failures
- ▶ $\approx$ thousands of hard drive failures slow disks, bad memory, misconfigured machines, flaky machines, etc.
- ▶ Long distance links: wild dogs, sharks, dead horses, drunken hunters, etc.



Big Data

- ▶ IDC study estimates the Global Datasphere to be 221 Zettabytes (221 000 000 000 TB) in 2026
- ▶ The amount of data is estimated to grow to 700 Zettabytes by 2030.
- ▶ Data comes from: Video, digital images, sensor data, biological data, Internet sites, social media, Internet of Things, generative AI, . . .
- ▶ Some examples:
 - ▶ Netflix is collecting 1 PB of data per month from its video service user behavior
 - ▶ Rovio (Angry Birds game) was collecting in the order of 1 TB of data per day of games logs
- ▶ The problem of such large data massed, termed **Big Data** calls for new approaches to analyzing these data masses



Example: Simple Word Search

- ▶ Example: Suppose you need to search for a word in a 2TB worth of text that is found only once in the text mass using a compute cluster
- ▶ Assuming 100MB/s read speed, in the worst case reading all data from a single 2TB disk takes ≈ 5.5 hours
- ▶ If 100 hard disks can be used in parallel, the same task takes less than four minutes
- ▶ Scaling using **hard disk parallelism** is one of the design goals of scalable Big Data Platforms in the cloud



Scalable Big Data Platforms

- ▶ For large enough data sets we must use hard disk parallelism to do processing in reasonable time
- ▶ MapReduce is a programming paradigm developed at Google for scalable batch processing, initially developed to compute the Web search indexes for Google
- ▶ The Apache Spark system is a new framework which is currently replacing MapReduce as the distributed Big Data processing system of choice
- ▶ A high performance fault tolerant distributed file system is needed to support either Spark or MapReduce, for example Google GFS, or its open source clone Hadoop distributed file system (HDFS)
- ▶ Python interface to the Spark framework called PySpark framework will be used in the home exercises of the course

